

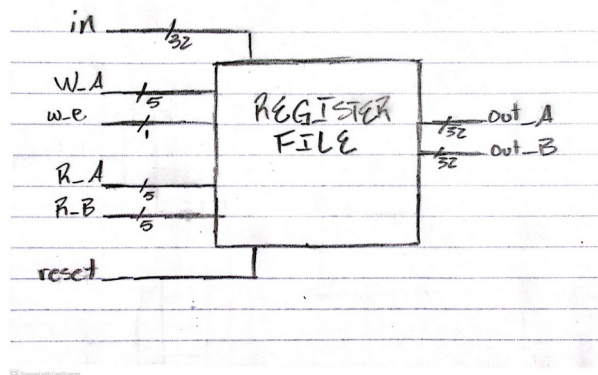
CprE 381, Computer Organization and Assembly-Level Programming

Lab 2 Report

Student Name _____ Isaiah Steele _____

Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 2 (a)] Draw the interface description (i.e., the “symbol” or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?



A complete register file requires:

Required for writing:

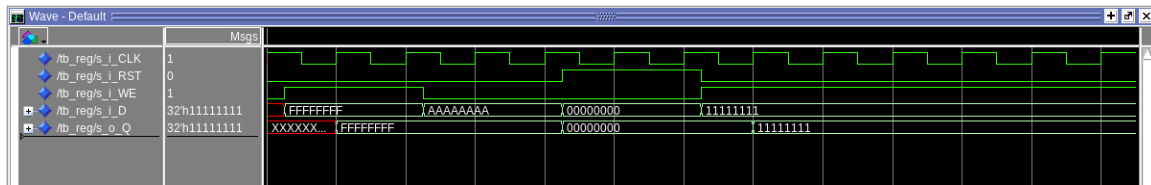
- 32-bit input bus (which can set the value of a register in the register file)
- 5-bit write address bus (5 bits can specify one of 32 registers) to select a register to write to
- Write enable line that determines whether the value in the input should be written to the register specified by the write address bus.

Required for reading:

- 32-bit output bus A that carries the output of one register
- 32-bit output bus B that carries the output of another register
- 5-bit read address bus A to specify what address the output bus A uses.
- 5-bit read address bus B to specify what address the output bus B uses.

[Part 2 (b)] Create an N-bit register using this flip-flop as your basis.
Complete.

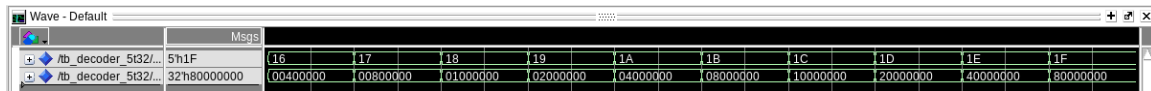
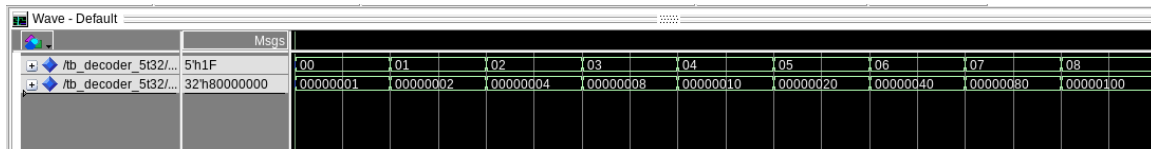
[Part 2 (c)] Waveform.



[Part 2 (d)] What type of decoder would be required by the RISC-V register file and why?

We need a 5-to-32 bit decoder for our register file. We have 32 registers and want to be able to specify one of those registers with a value in binary. Five bits can encode 32 different values, so we will encode our register with a 5 bit value and decode it into a 32-bit one-hot-encoded bus.

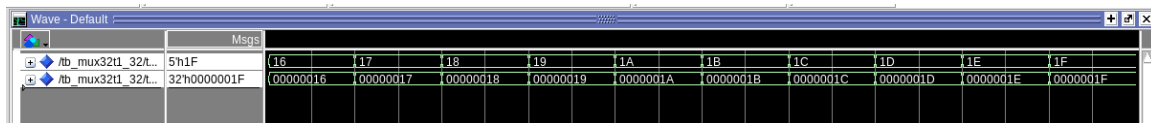
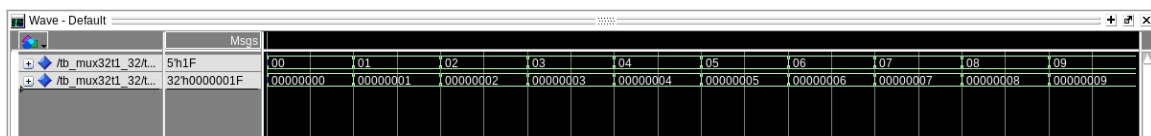
[Part 2 (e)] Waveform.



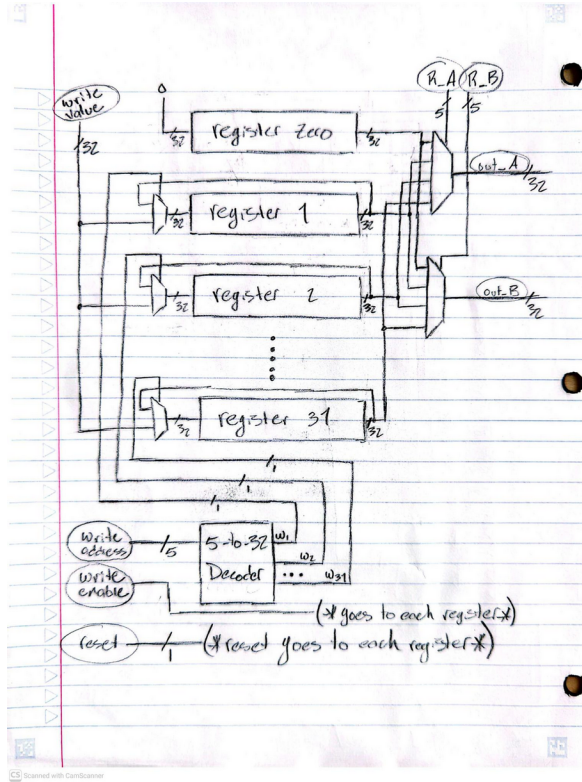
[Part 2 (f)] In your write-up, describe and defend the design you intend on implementing for the next part.

My design is structural. I have 32 incoming 32-bit buses and five stages of 32-bit 2:1 muxes. Each stage has an associated select line which runs to each mux in that stage and which cuts the used muxes in half, ending with a single selected 32-bit bus. This is not a clever design, but I know how it works and therefore easy for me to debug.

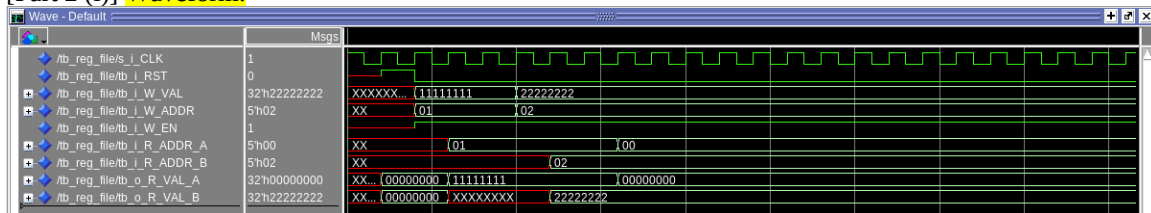
[Part 2 (g)] Waveform.



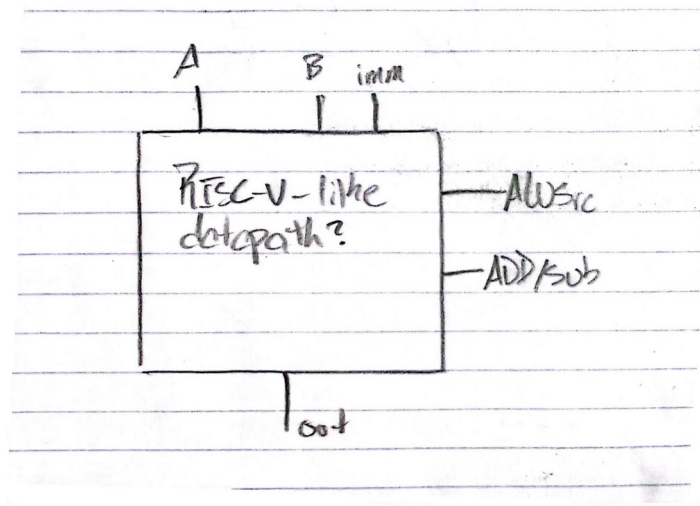
[Part 2 (h)] Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.



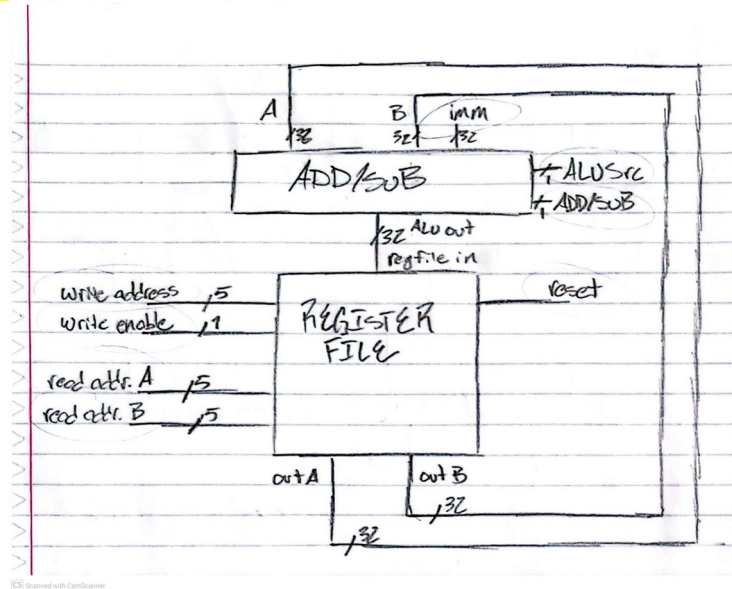
[Part 2 (i)] **Waveform.**



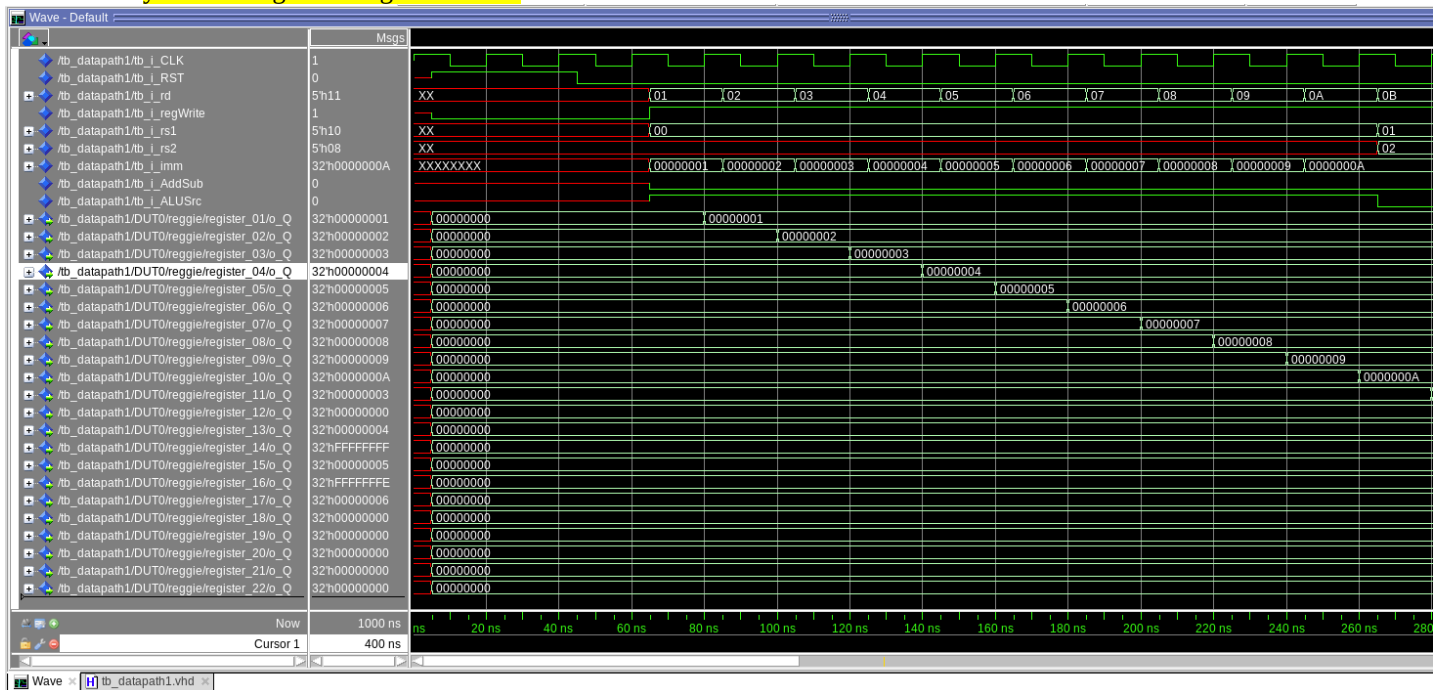
[Part 3 (b)] **Draw a symbol for this RISC-V-like datapath.**

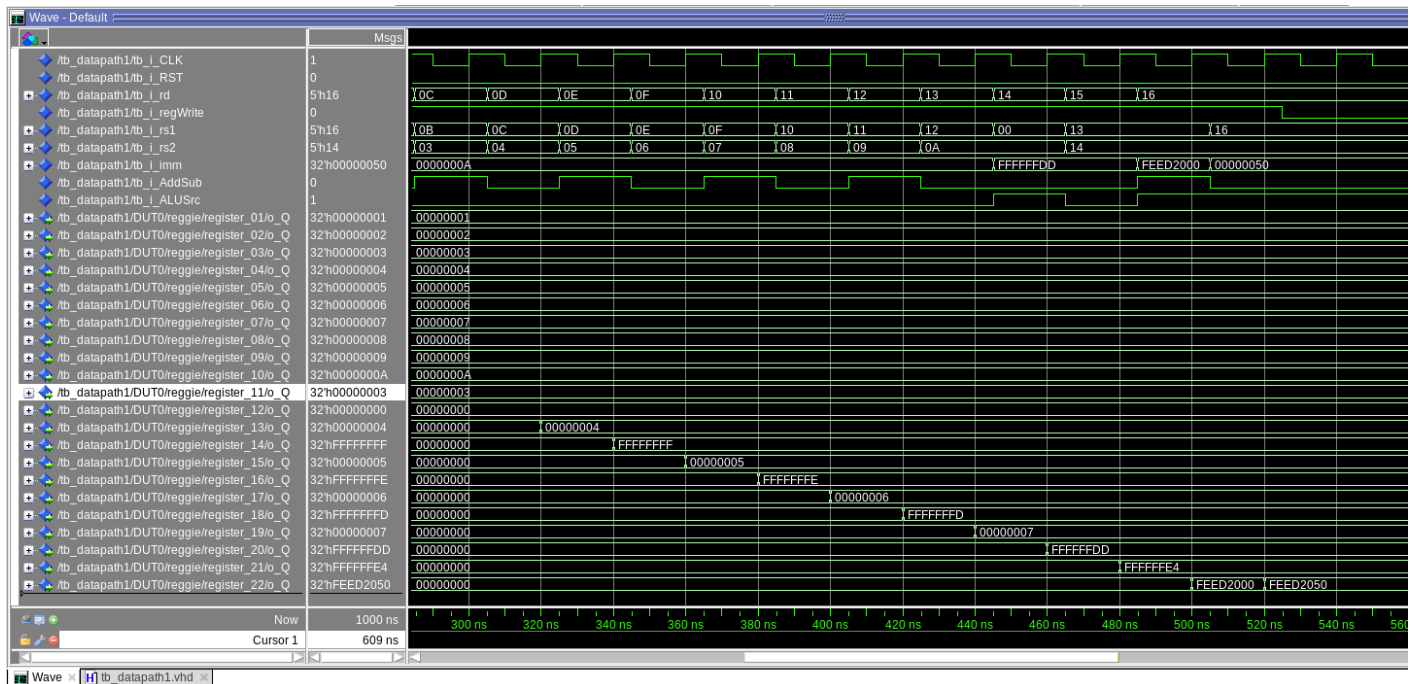


[Part 3 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



[Part 3 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final state of the register file should be and provide a mapping of the registers used in your drawing to the register names.





Each register (1 – 22) shown here maps to the RISC-V registers X1 – X22. Below is the expected final register state (because we were told in lecture that we are targeting RARS as our truth for our implementation):

Registers		
Floating Point		Control and Status
Name	Number	Value
zero	0	0x00000000
ra	1	0x00000001
sp	2	0x00000002
gp	3	0x00000003
tp	4	0x00000004
t0	5	0x00000005
t1	6	0x00000006
t2	7	0x00000007
s0	8	0x00000008
s1	9	0x00000009
a0	10	0x0000000A
a1	11	0x00000003
a2	12	0x00000000
a3	13	0x00000004
a4	14	0xffffffff
a5	15	0x00000005
a6	16	0xffffffffe
a7	17	0x00000006
s2	18	0xffffffffd
s3	19	0x00000007
s4	20	0xffffffffd
s5	21	0xffffffffe4
s6	22	0xfeed2050
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400060

[Part 4 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).
DATA_WIDTH: This generic port specifies a data width. It can be customized per instantiation, and defines the size of a word in the memory.

ADDR_WIDTH: This generic port specifies the size of an address to the memory in the memory module. It also defines the total size of memory available (all possible addresses given the data width are addressable).

BYTE_WIDTH: This generic port specifies the size of a byte in memory. Four bytes make up a word, the length of which is given in DATA_WIDTH.

clk: This port carries the clock signal into the memory module.

addr: This port carries an address into the memory module. The address describes what location/piece of memory the memory module will use (either read from or write to that address).

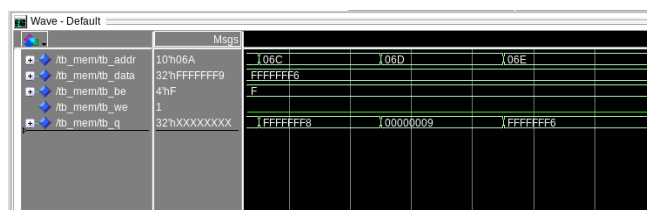
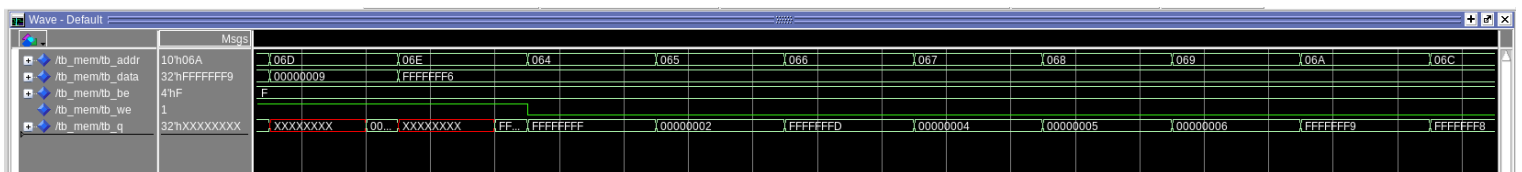
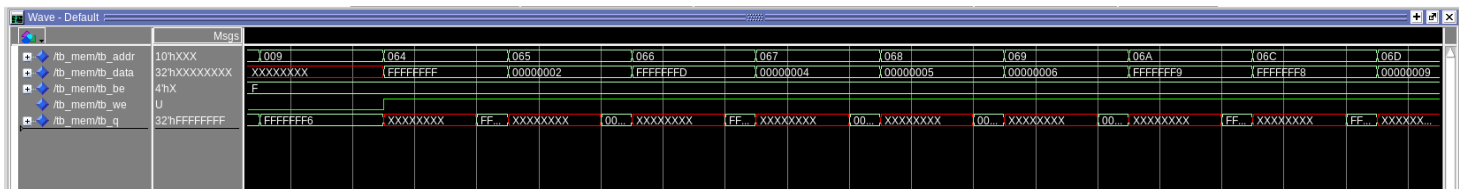
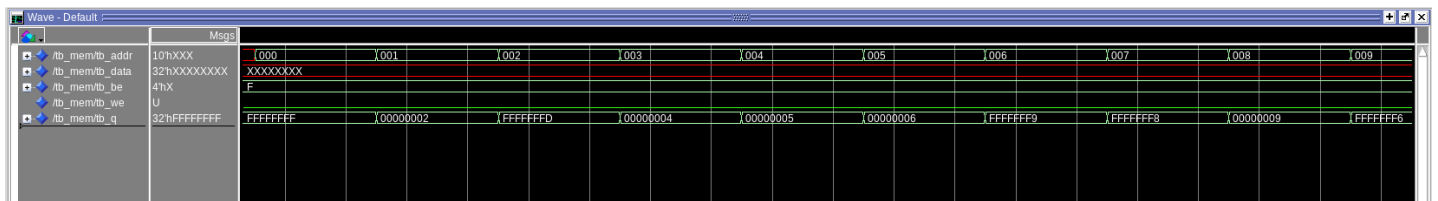
data: This port carries data into the memory module. It is used when writing to memory, and the data from this bus goes to the location given by the 'addr' bus.

be: this port stands for 'byte enable'. It acts as a secondary enable on the byte-level inside of a data word (in this case, 8-bit sections from a 32-bit word).

we: This port stands for 'write enable'. It determines whether the memory module reads or writes data.

q: This port is the "data out" bus. It carries data out of memory to other modules (the data which comes out is the data specified at the 'addr' address).

[Part 4 (c)] Waveforms and hex file.



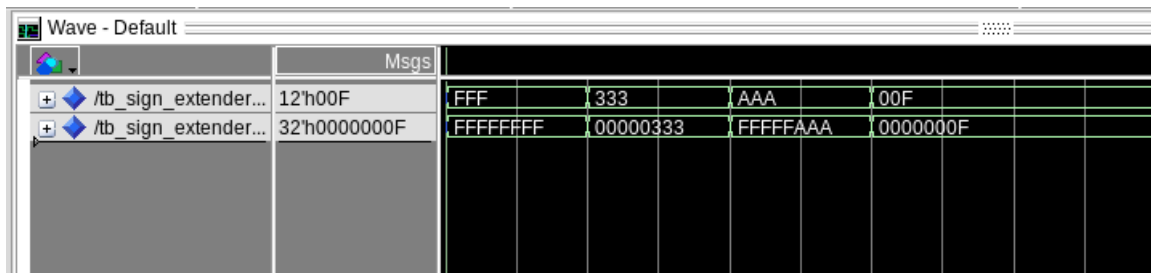
[Part 5 (a)] What are the RISC-V instructions that require some value to be sign extended? What are the RISC-V instructions that require some value to be zero extended? What are the RISC-V instructions that require some value to be appended with zero?

For this lab, the instructions that require some value to be sign extended are the *addi*, *lw*, and *sw*. Each of these have a 12-bit immediate that needs to be 32 bits in order to be used in the adder/subtractor unit. For this lab, the instruction that requires zeros appended to it is the *lui* instruction. The immediate is 20 bits and needs 0s appended so it can be

[Part 5 (b)] what are the different “extender” components that would be required by a RISC-V processor implementation (i.e., number of bits of input and sign vs zero extension vs zero appending)?

For this lab, the different “extender” components we will need are a sign extender that sign-extends a 12-bit immediate to 32 bits, and a zero appender which appends 12 zeros to a 20-bit immediate.

[Part 5 (d)] Waveform.



[Part 6 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?

The control signals that will need to be added are: *memory_write* line, *immediate_select*, and a *memory_to_register* line. The *immediate_select* decides what immediate extender we use, but the *memory_write* enables or disables write to memory, and the *memory_to_register* decides what input goes into the register file based on if we are moving data from memory to registers or not.

[Part 6 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.

In the screenshots above, similarly-colored rectangles show matching locations in memory. The red rectangles show the data loaded in the hex file. The green rectangles show the data loaded in the main part of the program, and the blue rectangles show the data stored at the end of the memory as the last action of the program.